GenSpark

TDD with pytest

In Week 1 you tested your Number Guessing Game by hand. You typed inputs, read the screen, and filled in a table. That teaches you what to test, but it is slow, and you have to repeat all of it every time you change the code. This article teaches you to write tests as code, so the computer runs them in a fraction of a second. It also introduces test-driven development, or TDD, a way of working in which you write the test first and the code second.

From Manual Test Plans to Automated Tests

Everything you learned about test plans carries over. Only the way you run them changes:

Week 1 manual test plan	Week 2 automated test
"Set `DEBUG_SECRET = 42`, guess 10"	Call `judge_guess(10, 42)` with the secret passed in
"Expected: says too low"	`assert ... == "too low"`
"Set `TIME_LIMIT = 5` and wait"	A fake clock that you move forward instantly
Read the screen and mark Yes or No	pytest prints PASSED or FAILED
Repeat everything by hand after each change	`pytest` reruns every test in a second

The hard part, deciding what to test and what the right answer is, is the same skill. The functions article already set this up: functions that take values and return values are easy to test, and that is why you kept `input()` and `print()` out of your logic.

What a Test Looks Like

A test is just a function whose name starts with `test_`. Inside it, you run some code and use `assert` to state what must be true:

python

```
def test_guess_below_secret_is_too_low():
    assert judge_guess(10, 42) == "too low"
```

`assert` means "this must be true." If the condition is true, the test passes. If it is false, the test fails and pytest tells you exactly what was different. That is the whole mechanism.

Installing pytest

pytest is a third-party package, so install it inside your project's virtual environment (the Number Guessing Game lab showed the steps):

bash

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install pytest
pytest --version
```

If you skip the virtual environment, a Homebrew Python on your Mac refuses with an "externally-managed-environment" error. Your prompt should show `(.venv)` while it is active.

Then save your packages to a file, so teammates and automated pipelines can install the same ones:

bash

```
pip freeze > requirements.txt
```

Project Layout

A common layout keeps your code in the project folder and your tests in a `tests` folder:

```
my_project/
    conftest.py
```

```
requirements.txt
board.py
ai.py
guess.py
tests/
    test_board.py
    test_ai.py
    test_guess.py
```

pytest finds tests by name:

- Test files must be named `test_something.py` (or `something_test.py`).
- Test functions must start with `test_`.
- Test classes, if you use them, must start with `Test`.

A file named `check_board.py`, or a function named `check_cell()`, is silently ignored. If pytest finds nothing, it prints `no tests ran` and exits with code 5.

Why the lab needs conftest.py

Your tests import your code with lines like `from board import Board`. But the tests live in the `tests` folder, and Python only looks for modules in certain places. If the project folder is not one of them, you get this:

```
tests/test_guess.py:1: in <module>
    from guess import judge_guess
E   ModuleNotFoundError: No module named 'guess'
```

An empty file named `conftest.py` in the project root fixes it. pytest adds the folder that contains `conftest.py` to Python's search path, so `from guess import ...` works. (Running `python3 -m pytest` also works, because that form adds your current folder. The empty `conftest.py` lets plain `pytest` work too.) `conftest.py` has a second job that you will meet shortly: it is where shared fixtures live.

Running Tests

Run these from the project folder, with the virtual environment active:

Command	What it does
`pytest`	Runs every test
`pytest -v`	Verbose: one line per test, with its name
`pytest -q`	Quiet: just dots and a summary
`pytest tests/test_board.py`	Runs one file
`pytest -k "timer"`	Runs only tests whose names contain `timer`
`pytest -x`	Stops at the first failure
`pytest --lf`	Reruns only the tests that failed last time
`pytest -s`	Shows `print()` output, which pytest normally hides

PyCharm can also run tests for you. With pytest installed, a green arrow appears next to each test function, and you can right-click a test file to run it. If PyCharm offers a different test runner, such as unittest, change the default test runner to pytest in Settings, under Tools, Python Integrated Tools. Menu names vary by version.

Reading the output

A passing run looks like this:

```
tests/test_guess.py ...                                                  [
3 passed in 0.01s
```

Each dot is a passing test. An `F` is a failing test, and an `E` is an error, meaning the test itself could not run, for example because of an import problem or a typo. A summary at the bottom tells you the counts. Your header lines, showing the platform and versions, will differ from the examples here.

assert in Detail

pytest rewrites your `assert` lines so failures show the actual values. You do not need special methods like `assertEqual`. Plain `assert` is enough:

python

```
assert board.cell(1, 1) == "X"
assert move in board.empty_cells()
assert len(board.empty_cells()) == 9
assert board.place_mark(0, 0, "X") is True
```

Three details:

- **Comparing floats.** Decimal math has tiny rounding errors, so `0.1 + 0.2 == 0.3` is `False`. Use `pytest.approx`: `assert 0.1 + 0.2 == pytest.approx(0.3)`.
- `is True` **versus a plain** `assert`. `assert board.place_mark(...)` passes for any truthy value. `assert board.place_mark(...) is True` insists on exactly `True`, which is stricter and often clearer for methods that promise a boolean.
- **Do not put parentheses around the whole assertion.** `assert (1 == 2, "message")` is a tuple, which is always true, so the test passes even though it should fail. pytest warns you with `assertion is always true, perhaps remove parentheses?`. Write `assert 1 == 2, "message"`.

Test-Driven Development

TDD is a three-step loop, repeated in very small steps:

1. **Red.** Write a test for a behavior that does not exist yet. Run it and watch it fail.
2. **Green.** Write the simplest code that makes the test pass. Not the cleverest, just enough.
3. **Refactor.** Clean up the code, with the passing tests as a safety net. The tests must stay green.

Then pick the next small behavior and repeat.

Why watch the test fail first? A test that has never failed might not test anything. Seeing it go red proves it can detect a problem. The loop also keeps you honest, because you can only write code that a test asked for, and it leaves you with a full set of tests as a side effect.

A worked example: judge_guess

Start with the guessing game's decision function from the functions article. Write the test first, before the function exists. Create `tests/test_guess.py` :

python

```
from guess import judge_guess


def test_guess_below_secret_is_too_low():
    assert judge_guess(10, 42) == "too low"
```

Red. Run `pytest` . There is no `guess.py` yet, so the run fails before any test runs:

```
tests/test_guess.py:1: in <module>
    from guess import judge_guess
E   ModuleNotFoundError: No module named 'guess'
```

That is a legitimate first red. Now create `guess.py` with the simplest thing that could pass:

python

```
def judge_guess(guess, secret):
    return "too low"
```

Green. Run `pytest -v` :

```
tests/test_guess.py::test_guess_below_secret_is_too_low PASSED           [

1 passed in 0.00s
```

It cheats, because the function always says "too low," but that is fine. The next test will force it to do more. Add a second test to the file:

python

```
def test_guess_above_secret_is_too_high():
    assert judge_guess(90, 42) == "too high"
```

Red. Run `pytest`:

```
    def test_guess_above_secret_is_too_high():
>       assert judge_guess(90, 42) == "too high"
E       AssertionError: assert 'too low' == 'too high'
E
E         - too high
E         + too low

tests/test_guess.py:9: AssertionError
FAILED tests/test_guess.py::test_guess_above_secret_is_too_high
1 failed, 1 passed
```

Read a failure from the `>` line. It points to the assertion that broke, and the `E` lines show the actual value on the left (`'too low'`) and the expected one on the right (`'too high'`). Make it pass:

python

```
def judge_guess(guess, secret):
    if guess < secret:
        return "too low"
    return "too high"
```

Add a third test for a correct guess:

python

```
def test_guess_equal_to_secret_is_correct():
    assert judge_guess(42, 42) == "correct"
```

It goes red again (`assert 'too high' == 'correct'`), because the function treats every non-low guess as high. Fix it:

python

```
def judge_guess(guess, secret):
    """Return "too low", "too high", or "correct"."""
    if guess < secret:
        return "too low"
    if guess > secret:
        return "too high"
    return "correct"
```

Green. All three pass:

```
tests/test_guess.py::test_guess_below_secret_is_too_low PASSED           [
tests/test_guess.py::test_guess_above_secret_is_too_high PASSED          [
tests/test_guess.py::test_guess_equal_to_secret_is_correct PASSED        [

3 passed in 0.01s
```

The code grew one behavior at a time, and each behavior has a test that proves it.

A second example, with refactoring

Suppose the Tic-Tac-Toe `Board` needs a way to count how many cells hold a given mark. Write the test first. Create `tests/test_count.py` :

python

```
from board import Board


def test_count_marks():
    board = Board(3)
    board.place_mark(0, 0, "X")
    board.place_mark(1, 1, "X")
```

```
    board.place_mark(2, 2, "O")
    assert board.count("X") == 2
    assert board.count("O") == 1
```

Red.

```
>       assert board.count("X") == 2
E       AttributeError: 'Board' object has no attribute 'count'
```

Green. Add the plainest version you can think of to the `Board` class:

python

```
    def count(self, mark):
        """Return how many cells hold mark."""
        total = 0
        for row in self._grid:
            for cell in row:
                if cell == mark:
                    total += 1
        return total
```

The test passes. **Refactor.** Now that you have a safety net, shorten the code:

python

```
    def count(self, mark):
        """Return how many cells hold mark."""
        return sum(row.count(mark) for row in self._grid)
```

Run `pytest` again. It still passes, so you know the cleanup did not change the behavior. That is the power of refactoring with tests. You can improve the design without fear.

TDD when you find a bug

The same idea works for bugs. When you discover a bug, **first write a test that reproduces it and fails**. Then fix the code. The test stays in your suite forever as a **regression test**, so that bug can

never quietly return. The off-board tests for the `Board` below exist because of the negative-index bug from the lists article.

Writing Good Tests

Arrange, Act, Assert. Most tests have three parts:

python

```
def test_cannot_place_on_taken_cell():
    board = Board(3)                          # Arrange: set up
    board.place_mark(0, 0, "X")
    result = board.place_mark(0, 0, "O")      # Act: do the thing
    assert result is False                    # Assert: check the outcome
    assert board.cell(0, 0) == "X"
```

Name tests like sentences. `test_cannot_place_on_taken_cell` tells you what broke without opening the file. Names like `test1` and `test_board` tell you nothing. Long names are fine.

One behavior per test. If a test checks five things and fails, you do not know which one is broken. A few related asserts about one behavior are fine.

Make tests independent. Each test must set up what it needs and not rely on another test having run first. pytest may run them in any order.

Make tests deterministic. The same test must give the same result every time. That rules out unseeded randomness, the real clock, and the real network.

Make tests fast. A test that sleeps for 60 real seconds is a test nobody will run.

Test behavior, not internals. Test what a method promises (`place_mark` returns `False` for a taken cell), not how it does the work. Then you can rewrite the inside without touching the tests.

Cover the same kinds of cases as your test plan. Normal cases, boundaries (just below, on, and just above an edge), invalid input, and sequences of steps.

Avoid logic in tests. A test with its own `if` statements and loops can contain its own bugs. Keep tests flat and obvious.

Fixtures: Fresh Setup for Each Test

Many tests start with the same setup, such as a fresh board. A **fixture** is a function that prepares something and hands it to tests. Mark it with `@pytest.fixture`, and any test that lists its name as a parameter receives the result:

python

```
import pytest

from board import Board


@pytest.fixture
def board():
    """A fresh, empty 3x3 board for each test."""
    return Board(3)


def test_new_board_is_empty(board):
    assert len(board.empty_cells()) == 9


def test_place_mark_puts_mark_on_cell(board):
    assert board.place_mark(1, 1, "X") is True
    assert board.cell(1, 1) == "X"
```

pytest sees the parameter `board`, finds the fixture with that name, runs it, and passes in the result. By default each test gets its **own new** board, so one test's moves can never leak into another. If several test files need the same fixture, move it into `conftest.py` and it becomes available everywhere.

Parametrized Tests: Many Cases, One Test

When you want to run the same check with many inputs, `@pytest.mark.parametrize` avoids copying the test:

python

```
@pytest.mark.parametrize("row, col", [(-1, 0), (0, -1), (3, 0), (0, 3)])
def test_off_board_moves_are_rejected(board, row, col):
    assert board.place_mark(row, col, "X") is False
    assert len(board.empty_cells()) == 9
```

The first argument names the parameters, and the list supplies one set of values per run. pytest runs the test four times and reports each separately:

```
tests/test_board.py::test_off_board_moves_are_rejected[-1-0] PASSED
tests/test_board.py::test_off_board_moves_are_rejected[0--1] PASSED
tests/test_board.py::test_off_board_moves_are_rejected[3-0] PASSED
tests/test_board.py::test_off_board_moves_are_rejected[0-3] PASSED
```

This is a direct translation of a test plan table. Each tuple is one row. It also makes boundary testing cheap: just add another pair of values.

Testing That Errors Happen

Sometimes the correct behavior is to raise an error. Use `pytest.raises`:

python

```
def test_board_smaller_than_three_is_invalid():
    with pytest.raises(ValueError):
        Board(2)
```

The test passes only if the code inside the `with` block raises a `ValueError`. If no error occurs, or a different error occurs, the test fails. To check the message too, add `match`, which looks for that text in the error message:

python

```
with pytest.raises(ValueError, match="at least 3"):
    Board(2)
```

Testing Randomness

Random behavior sounds untestable, but you have two tools.

1. Pass in a seeded generator. The `EasyAI` from the last article accepts a `random.Random` object. A seed fixes its sequence, so tests become repeatable:

python

```
import random

from ai import EasyAI
from board import Board


def test_easy_ai_with_same_seed_makes_same_choice():
    board = Board(3)
    first = EasyAI(rng=random.Random(7)).choose_move(board, "O")
    second = EasyAI(rng=random.Random(7)).choose_move(board, "O")
    assert first == second
```

2. Test the rule, not the exact value. You cannot say which cell an easy AI will pick, but you can say it must be *empty*. Try many seeds and check the rule every time:

python

```
def test_easy_ai_only_picks_empty_cells():
    board = Board(3)
    for r, c in [(0, 0), (0, 1), (1, 1), (2, 2)]:
        board.place_mark(r, c, "X")
    for seed in range(50):
        move = EasyAI(rng=random.Random(seed)).choose_move(board, "O")
        assert move in board.empty_cells()
```

Compare this with Week 1. There you set `DEBUG_SECRET = 42` and had to remember to change it back. Here the secret number or generator is simply a **parameter**, so tests supply what they need and the real program never changes. This is the same idea as the `secret=None` option in the functions article.

Testing Time

You do not want a test that waits for 15 real seconds. The trick is to let the code receive its clock as a parameter, with the real clock as the default. This `MoveTimer` tells you if a move took too long:

python

```
import time


class MoveTimer:
    """Tells whether a move took longer than the time limit."""

    def __init__(self, limit, clock=time.time):
        self._limit = limit
        self._clock = clock
        self._started = None

    def start(self):
        self._started = self._clock()

    def is_expired(self):
        return self._clock() - self._started > self._limit
```

In the real game, `MoveTimer(15)` uses `time.time`. In a test, you pass in a **fake clock** that you control:

python

```
from movetimer import MoveTimer


class FakeClock:
    """A clock the test controls."""

    def __init__(self):
        self._now = 0.0
```

```
        def advance(self, seconds):
            self._now += seconds

        def time(self):
            return self._now


    def test_move_within_limit_is_not_expired():
        clock = FakeClock()
        timer = MoveTimer(limit=15, clock=clock.time)
        timer.start()
        clock.advance(10)
        assert timer.is_expired() is False


    def test_move_over_limit_is_expired():
        clock = FakeClock()
        timer = MoveTimer(limit=15, clock=clock.time)
        timer.start()
        clock.advance(15.1)
        assert timer.is_expired() is True


    def test_move_exactly_at_limit_is_not_expired():
        clock = FakeClock()
        timer = MoveTimer(limit=15, clock=clock.time)
        timer.start()
        clock.advance(15)
        assert timer.is_expired() is False
```

`clock.advance(15.1)` moves the pretend time forward instantly, so the whole test runs in a fraction of a second. The third test checks the **boundary** from Week 1: exactly at the limit is still allowed, because the rule is "longer than 15 seconds." Boundary tests catch the `<` versus `<=` mistakes that the testing article warned about.

Testing a Family of Classes

Polymorphism makes testing easy. Every AI promises to return a legal move, so one parametrized test can check all of them:

python

```
from ai import AIPlayer, EasyAI, FirstEmptyAI


@pytest.mark.parametrize("ai", [FirstEmptyAI(), EasyAI(rng=random.Random(1
def test_every_ai_returns_a_legal_move(ai):
    board = Board(3)
    row, col = ai.choose_move(board, "O")
    assert board.place_mark(row, col, "O") is True


def test_first_empty_ai_picks_first_free_cell():
    board = Board(3)
    board.place_mark(0, 0, "X")
    assert FirstEmptyAI().choose_move(board, "O") == (0, 1)


def test_ai_player_cannot_be_created_directly():
    with pytest.raises(TypeError):
        AIPlayer()
```

When you add a new AI level, put it in that list and it gets the same test for free. A deliberately predictable opponent like `FirstEmptyAI` is also useful for testing the game itself, because the opponent's moves become known. (When pytest has only objects to label, it names the runs `ai0`, `ai1`, and so on.)

Keep Input and Output Out of Your Logic

Notice that none of the tests above call `input()` or read the screen. They call methods and check return values. That is only possible because the logic classes do not use `input()` and `print()`, the design rule from the functions and classes articles. The thin outer layer that does talk to the player, `main.py`, is small enough to check by playing the game. pytest does have tools for faking input and capturing printed output, which you may meet later, but the better design is to avoid needing them.

Tests in the Bigger Picture

- **Tests run automatically.** In the CI/CD pipelines article, a GitHub Actions workflow runs `pytest` on every pull request. A failing test shows a red X on the pull request and can

block the merge.

- **Tests make AI-generated code safer.** When an AI assistant writes code for you, a test you wrote first is your specification. If the code passes, you have evidence it does the right thing. You will build on this in Week 4.
- **Tests do not prove there are no bugs.** They prove that the cases you thought of pass. Passing tests with no boundary or invalid-input cases give false comfort.
- **More tests are not always better tests.** Twenty copies of the same check add nothing. Aim for tests that cover different behaviors and different edges.

Common Mistakes

Mistake	What happens	Fix
Test file or function not starting with `test_`	`no tests ran`	Rename it
No `conftest.py` and plain `pytest`	`ModuleNotFoundError` when importing your code	Add an empty `conftest.py` to the project root
Running pytest without the virtual environment active	`pytest: command not found`	Activate `.venv`, or use `python -m pytest`
Writing the code first, then a test that matches it	The test confirms the bug	Write the test first and watch it fail
Never seeing the test fail	It may not test anything	Break the code on purpose once to be sure
Tests that depend on each other	Pass or fail depending on order	Make each test set up its own data
Using real randomness in a test	Passes sometimes and fails sometimes (flaky)	Pass in a seeded `random.Random` or test the rule
Using the real clock or `time.sleep`	Slow tests	Pass in a fake clock
`assert (a == b,`		`assert a == b,`

`"message")`	Always passes	`"message"`
Comparing floats with `==`	Rounding failures	Use `pytest.approx`
One giant test checking everything	Hard to tell what broke	One behavior per test
Testing private details	Tests break when you refactor	Test public behavior
Skipping boundary and invalid-input cases	Bugs hide at the edges	Add just-below, on, and just-above values
Sharing one board across tests at module level	One test's moves affect others	Use a fixture to build a fresh one
Reading only the last line of a failure	You miss the actual and expected values	Read the `>` line and the `E` lines

Try It Yourself

Create a project folder with a virtual environment, an empty `conftest.py`, and a `tests` folder. Try these:

1. Install pytest in your virtual environment and run `pytest --version`. Run `pytest` with no tests and note the output and what "no tests ran" means.
2. Follow the `judge_guess` example from this article step by step. Write each test first, run it, watch it fail, and then write the code. Do not skip the red steps.
3. Break `judge_guess` on purpose by changing `<` to `<=`. Which test fails? Read the failure output and explain it in a sentence. Then fix it.
4. Remove `conftest.py` and run plain `pytest`. What error do you get? Then run `python3 -m pytest`. Put `conftest.py` back.
5. Name a test file `check_guess.py` and run `pytest`. What happens? Rename it.
6. Using TDD, write `is_even(number)`. Write at least five tests first, including 0, a negative number, and a large number.
7. Using TDD, write `calculate_grade(score)` from the decisions article. Use `parametrize` to test every boundary: 59, 60, 69, 70, 79, 80, 89, 90, 100.

8. Using TDD, write a function that rejects scores below 0 or above 100 by raising `ValueError`. Test it with `pytest.raises` and `match`.

9. Type in the `Board` class from the classes article and write fixture-based tests for `place_mark`, `empty_cells`, `is_full`, and `copy`.

10. Add `count(mark)` to the `Board` using the red, green, refactor loop. Write the loop version first, then refactor to `sum(...)` and confirm the tests still pass.

11. Find the negative-index bug: remove the range check from `place_mark`, run your off-board tests, and see which fail. Put the check back.

12. Write tests for `EasyAI` that use a seeded generator. Confirm that the same seed gives the same move, and that 100 different seeds never choose a taken cell.

13. Type in `MoveTimer` and `FakeClock` and write the three timer tests. Then change `>` to `>=` in `is_expired` and see which test catches it.

14. Convert five rows of your Number Guessing Game test plan into pytest tests. Which rows were easy to automate? Which were hard, and why?

15. Run `pytest -x`, `pytest -k "board"`, and `pytest --lf`. Describe what each one changes. Make one test fail first so `--lf` has something to rerun.

16. Write a test that fails because of a bug you find in your own Week 1 code. Fix the code and keep the test.

Why This Matters for an FDE

On a client engagement, you will change code you do not fully understand, and you will do it under time pressure. A good test suite is what lets you do that without fear: if you break something, you find out in seconds, not from an angry user. Tests are also how you prove to a client that your work does what you said, and they let your teammates change your code safely after you move on. TDD is increasingly central to AI-assisted development, where tests act as the specification that tells you whether generated code can be trusted.

Comprehension Check

1. What are the three steps of the TDD loop, and why do you watch the test fail before writing the code?

2. A test file is called `check_board.py` and contains `def check_cell():` . Will pytest run it? Why or why not?
3. What does an empty `conftest.py` in the project root do for your tests?
4. What is the difference between a failing test (`F`) and an error (`E`)?
5. Why is `assert 0.1 + 0.2 == 0.3` a bad test, and what should you use instead?
6. What does a fixture do, and why does each test get its own fresh board?
7. How does `@pytest.mark.parametrize` relate to a row in a test plan table?
8. How do you test that code raises an error? Write the pattern.
9. You cannot know which cell a random AI picks. Name two ways to test it anyway.
10. Why does `MoveTimer` accept a `clock` parameter, and how does that avoid waiting 15 real seconds in a test?
11. What is a regression test, and when do you write one?
12. A teammate says, "all my tests pass, so my code has no bugs." What is wrong with that claim?

← Previous: OOP: Inheritance & Polymorphism

Next: SDLC Foundations: Requirements Gathering and System Design Basics →

0 comments so far

6

Add a comment...

Comment cannot be empty

Comment

No comments yet. Be the first to start the discussion!